

DOCUMENTATION UTILISATEUR

Version de la documentation — 2026-07-21

Sommaire Animate JS

1.	Vue d'ensemble de l'interface
Éditeur d'animation vectorielle image par image, inspiré d'Adobe	
2.	La scène et les outils de dessin
Animate, entièrement en JavaScript vanilla — de la scène de dessin	
jusqu'à l'export d'objets de jeu réutilisables.	La plume Bézier et l'édition de points
4.	La timeline : calques, images clés, tweens
5.	Symboles et bibliothèque (Graphic / MovieClip)
6.	Annuler / rétablir
7.	Panneaux réductibles et redimensionnables
8.	Enregistrer, ouvrir, exporter
9.	Exploiter les MovieClips pour faire des jeux en JavaScript vanilla

1. Vue d'ensemble de l'interface

La fenêtre d'Animate JS est découpée en cinq zones :

Zone	Rôle
Barre de menu (haut)	Annuler/rétablir, Nouveau, Ouvrir, Enregistrer JSON, Exporter HTML, propriétés du document (nom, largeur, hauteur, images/seconde, nombre d'images, couleur de fond).
Barre d'outils (gauche)	Outils de sélection et de dessin, couleurs de remplissage/contour.
Scène (centre)	Le canvas de travail — dessine, sélectionne, transforme les objets ici.
Panneau latéral (droite)	Bibliothèque (symboles) en haut, Propriétés de l'objet sélectionné en bas.
Timeline (bas)	Calques, images clés, lecture, tweens, étiquettes d'image.

Quand tu double-cliques sur une instance de symbole (ou un élément de la bibliothèque), la scène bascule en **édition isolée** de ce symbole : un bandeau bleu apparaît en haut de la scène — cliquer dessus ramène à la scène principale.

2. La scène et les outils de dessin

La barre d'outils, de haut en bas :

Outil	Raccourci	Usage
❖ Sélection	V	Sélectionner, déplacer, redimensionner et pivoter (poignées).
◇ Sous-sélection	A	Éditer les points d'ancrage/poignées d'une courbe existante. Voir section 3.
□ Rectangle	R	Cliquer-glisser pour dessiner.
○ Ellipse	O	Cliquer-glisser pour dessiner.
/ Ligne	L	Cliquer-glisser pour un segment droit.
🖋 Plume	P	Courbes de Bézier. Voir section 3.
T Texte	T	Clique sur la scène pour poser un bloc de texte (modifiable ensuite dans le panneau Propriétés).

Avec l'outil Sélection : Suppr efface la sélection, le glisser déplace, les poignées du cadre redimensionnent/pivotent. Les couleurs de remplissage et de contour (en bas de la barre d'outils) s'appliquent aux prochaines formes dessinées, ou à la sélection courante via le panneau Propriétés.

3. La plume Bézier et l'édition de points

L'outil Plume () dessine des courbes de Bézier, comme dans Illustrator ou Animate :

- **Clic simple** : ajoute un point anguleux (pas de courbure).
- **Clic-glissé** : ajoute un point lisse avec deux poignées symétriques — la courbe s'incurve de part et d'autre.
- **Clic près du premier point** : referme le tracé (forme remplie).
-  ou double-clic : termine un tracé ouvert (juste un contour).
-  : annule le tracé en cours.

Une fois la courbe créée, l'outil **Sous-sélection** () permet de cliquer dessus pour faire apparaître :

- les **points d'ancrage** (cercles blancs, le premier en jaune) — glisse-les pour déplacer un point ;
- les **poignées de courbure** (petits cercles orange, reliés par une fine ligne) — glisse-les pour ajuster la forme de la courbe. Sur un point "lisse", les deux poignées bougent en miroir automatiquement.

Limite connue : il n'est pas possible d'ajouter une poignée à un point anguleux existant depuis l'outil Sous-sélection — seul le clic-glissé au moment de la création en crée une.

4. La timeline : calques, images clés, tweens

Calques

Chaque calque a son propre contenu, indépendant des autres. Boutons de la timeline : **+** (nouveau calque), **🗑** (supprimer), et sur chaque ligne de calque : **👁** (visibilité), **🔒 / 🔓** (verrouillage), double-clic sur le nom pour renommer.

Images clés

Action	Raccourci	Effet
F6	F6	Insère une image clé au niveau de la tête de lecture, en clonant le contenu de l'image précédente.
F7	F7	Insère une image clé <i>vide</i> (rien ne s'affiche jusqu'à la prochaine image clé).
X	—	Supprime l'image clé courante.
Lecture/Pause	Espace	Joue l'animation en boucle sur toute la longueur de la scène (ou du symbole en cours d'édition).

Un point plein marque une image clé avec contenu, un cercle creux une image clé vide. Cliquer sur la règle en haut (ou glisser dessus) déplace la tête de lecture.

Tween de mouvement

Le bouton **↔** active/désactive l'interpolation entre l'image clé courante et la suivante. S'il n'y a pas encore d'image clé suivante, une nouvelle est créée automatiquement 12 images plus loin (contenu cloné) et la tête de lecture s'y déplace, prête à ce que tu repositionnes les objets pour définir le mouvement. Le panneau Propriétés propose 4 courbes d'accélération (linéaire, accélération, décélération, les deux).

Morphing de courbes : si une courbe (dessinée à la plume) tweenée a *exactement le même nombre de points* sur l'image de départ et celle d'arrivée, ses points sont interpolés un à un — la forme se déforme réellement. Sinon, elle bouge comme un bloc rigide (position/rotation/échelle), sans se déformer. L'appariement se fait par index (1^{er} point avec 1^{er}, 2^e avec 2^e...), sans "shape hints" pour guider l'appariement comme dans Animate.

Étiquettes d'image (labels)

Le champ "Étiquette" de la timeline nomme l'image courante (ex. **walk**, **jump**) — affichée avec un petit **🏷** sur la règle. Ces noms deviennent utilisables avec **gotoAndPlay('walk')** une fois le symbole exporté (voir section 9).

5. Symboles et bibliothèque (Graphic / MovieClip)

Un symbole est un contenu réutilisable, comme dans Animate. Deux types :

Type	Comportement
GRAPHIC	Sa timeline interne reste synchronisée sur l'image de la scène parente (image N du parent → image N mod (nb d'images du symbole)).
MOVIECLIP	Sa timeline interne est indépendante : elle avance toute seule pendant la lecture, comme un mini-programme autonome — c'est le type à utiliser pour des objets de jeu (section 9).

Créer un symbole

- **+ Nouveau symbole** (`Ctrl+F8`) : crée un symbole vide et bascule directement en édition isolée dessus (scène vierge dédiée) — pratique pour dessiner un personnage ou un objet de zéro.
- **Convertir en symbole** (`F8`) : transforme la sélection courante (une ou plusieurs formes, éventuellement déjà animées) en symbole. Si les objets sélectionnés étaient déjà tweenés, l'animation est préservée après conversion.

Utiliser un symbole

Dans la bibliothèque, chaque symbole propose : **+** (poser une instance au centre de la scène), **✎** (renommer), **🎮** (exporter comme objet de jeu JS — section 9), **🗑** (supprimer). Double-clic sur un symbole (ou sur une instance dans la scène) pour l'éditer en isolation ; le bandeau en haut de la scène ramène à la scène principale.

6. Annuler / rétablir

Boutons ↶ **Annuler** / ↷ **Rétablir** dans la barre de menu, raccourcis `Ctrl+Z` / `Ctrl+Y` (`Ctrl+Shift+Z` fonctionne aussi pour rétablir). 15 niveaux d'historique. Seuls les changements réels du document sont suivis (formes, calques, images clés, symboles...) — la simple sélection ou le défilement de la timeline ne compte pas comme une action annulable.

7. Panneaux réductibles et redimensionnables

- Le bouton ▾ / ▸ en haut de la **timeline** et de la **bibliothèque** les réduit à leur simple barre de titre, pour libérer de la place.
- Une poignée verticale entre la scène et le panneau latéral permet de **glisser pour redimensionner** la largeur de la bibliothèque et du panneau Propriétés (ils partagent la même colonne).
- Ces réglages sont mémorisés dans le navigateur (localStorage) et retrouvés à la prochaine ouverture.

8. Enregistrer, ouvrir, exporter

Action	Résultat
Enregistrer JSON	Télécharge le document complet (calques, symboles, tout) dans un fichier <code>.json</code> , rechargeable via "Ouvrir..."
Ouvrir...	Recharge un fichier <code>.json</code> précédemment enregistré.
Exporter HTML	Génère un fichier <code>.html</code> autonome (aucune dépendance) qui rejoue toute la scène en boucle, avec le runtime <code>MovieClip</code> embarqué. Idéal pour partager un aperçu ou publier une animation simple.
 (par symbole, dans la bibliothèque)	Télécharge <code>animate-runtime.js</code> + <code><NomDuSymbole>.js</code> — un objet réutilisable dans du code JS, détaillé section 9.

9. Exploiter les MovieClips pour faire des jeux en JavaScript vanilla

9.1 Ce que tu obtiens en exportant un symbole

Le bouton  d'un symbole dans la bibliothèque télécharge deux fichiers :

- `animate-runtime.js` — le moteur de lecture, à garder **un seul exemplaire** par projet de jeu, quel que soit le nombre de symboles exportés.
- `<NomDuSymbole>.js` — une petite classe qui étend `MovieClip` et embarque les données du symbole (et de tout ce dont il dépend, s'il contient d'autres symboles imbriqués).

Important : les deux fichiers doivent rester dans le même dossier (le second fait `import { MovieClip } from './animate-runtime.js'`). Aucune autre dépendance n'est nécessaire — pas de Konva, pas de bundler, pas d'installation npm côté jeu.

9.2 API de la classe MovieClip

Membre	Description
<code>new MonClip({ x, y, rotation, scaleX, scaleY, opacity, loop })</code>	Toutes les propriétés sont optionnelles (défauts : position 0,0 / rotation 0 / échelle 1 / opacité 1 / <code>loop: true</code>).
<code>x</code> , <code>y</code> , <code>rotation</code> , <code>scaleX</code> , <code>scaleY</code> , <code>opacity</code> , <code>visible</code>	Propriétés lisibles/modifiables à tout moment — c'est ainsi qu'on positionne/anime le clip depuis le code du jeu.
<code>play()</code> / <code>stop()</code>	Démarre/arrête l'avancement automatique des images.
<code>gotoAndPlay(frame label)</code>	Saute à une image (numéro, base 0) ou à une étiquette nommée dans l'éditeur, puis joue.
<code>gotoAndStop(frame label)</code>	Idem mais met en pause sur cette image.
<code>currentFrame</code> / <code>frameCount</code>	Lecture seule — image courante et nombre total d'images.
<code>addEventListener('complete', fn)</code>	Déclenché quand un clip non bouclé (<code>loop:false</code>) atteint sa dernière image.
<code>addEventListener('loop', fn)</code>	Déclenché à chaque retour au début, pour un clip en boucle.
<code>update(dt)</code>	Fait avancer l'animation. <code>dt</code> en millisecondes écoulées depuis le dernier appel — à appeler une fois par image de ta boucle de jeu.
<code>draw(ctx)</code>	Dessine le clip sur un contexte Canvas 2D existant, en tenant compte de x/y/rotation/échelle/opacité.

9.3 Exemple minimal

```
import { HeroClip } from './Hero.js';

const canvas = document.getElementById('game');
const ctx = canvas.getContext('2d');

const hero = new HeroClip({ x: 150, y: 300 });
hero.gotoAndPlay('idle'); // étiquette posée dans l'éditeur

let lastTime = null;
function loop(time) {
  requestAnimationFrame(loop);
  if (lastTime === null) lastTime = time;
  const dt = time - lastTime;
  lastTime = time;

  hero.update(dt);

  ctx.clearRect(0, 0, canvas.width, canvas.height);
  hero.draw(ctx);
}
requestAnimationFrame(loop);
```

9.4 Piloter les états d'un personnage avec des labels

Pose des étiquettes de frame dans l'éditeur (section 4) — par exemple `idle` en image 0, `walk` en image 6, `jump` en image 18 — puis pilote-les depuis ton code :

```
function onInput(direction) {
  if (direction === 'left' || direction === 'right') {
    hero.scaleX = direction === 'left' ? -1 : 1; // retourner le sprite
    if (hero.currentFrame < 6) hero.gotoAndPlay('walk');
  } else {
    hero.gotoAndPlay('idle');
  }
}

function onJump() {
  hero.loop = false;
  hero.gotoAndPlay('jump');
  hero.addEventListener('complete', () => {
    hero.loop = true;
    hero.gotoAndPlay('idle');
  });
}
```

9.5 Plusieurs instances indépendantes

Chaque `new HeroClip(...)` a son propre état de lecture — utile pour des ennemis, projectiles, effets, etc. animés indépendamment à partir du même symbole :

```
const enemies = Array.from({ length: 5 }, (_, i) =>
  new EnemyClip({ x: 80 + i * 60, y: 200 })
);

function loop(dt) {
  for (const e of enemies) { e.update(dt); e.draw(ctx); }
}
```

9.6 Symboles imbriqués : Graphic vs MovieClip

Si un symbole exporté contient lui-même des instances d'autres symboles :

- les instances de type **GRAPHIC** restent visuellement synchronisées à l'image du parent (aucun état propre à gérer) ;
- les instances de type **MOVIECLIP** tournent sur leur propre horloge dès que le clip parent est mis à jour — exactement le comportement observé dans l'éditeur.

Le fichier exporté embarque automatiquement toutes les données des symboles dont il dépend : rien à exporter séparément pour les symboles imbriqués.

9.7 Intégration avec un moteur (Phaser, PixiJS, etc.)

Un **MovieClip** ne demande rien de plus qu'un contexte Canvas 2D : dans un moteur qui expose un canvas offscreen ou une texture dynamique, appelle `update(dt)` puis `draw(ctx)` sur ce contexte à chaque tick, comme dans l'exemple 9.3 — aucune intégration spécifique n'est nécessaire.

En résumé : dessine et anime dans Animate JS, convertis en symbole MovieClip, exporte-le depuis la bibliothèque (🎮), puis utilise la classe générée comme n'importe quel objet de jeu — `x / y` pour la position, `gotoAndPlay()` pour changer d'état, `update()` / `draw()` dans ta boucle. Aucune dépendance à installer.